



2

10

1500 sp

عملي

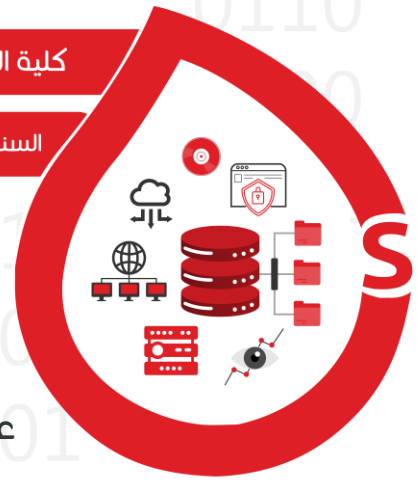
محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 3/11/2025

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Indexing

عملي مشترك



قواعد معطيات ٢

في مقرر قواعد البيانات 1 تعلمنا كيفية إنشاء قاعدة البيانات وكتابة الاستعلامات.

أما في قواعد البيانات 2، سنركز على تحسين الأداء، لأن حجم البيانات وطريقة الوصول إليها هما العاملان الأساسيان في سرعة الاستعلامات.

معظم العمل على أي نظام قاعدة بيانات يعتمد على عمليات القراءة (SELECT)، مثل عرض بيانات الموظفين، المنتجات، أو البحث عن سجلات معينة. هذه العمليات تمثل الجزء الأكبر من استهلاك النظام للوقت والموارد، خاصة عند التعامل مع جداول ضخمة.

لذلك، تحسين الأداء أمر بالغ الأهمية، لأنه يجعل قاعدة البيانات:

- سريعة في الاستجابة.
- أكثر كفاءة في استخدام الموارد مثل المعالج (CPU) والذاكرة (RAM).
- قابلة للتوسع مع نمو حجم البيانات يوميًا.

أحد الحلول الأساسية لتحسين الأداء هو الفهارس (Indexing)، التي تختصر الطريق للوصول إلى البيانات المطلوبة بدلاً من مسح الجدول بالكامل.

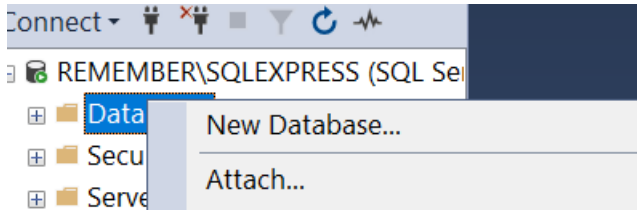
في المحاضرة القادمة سنتعرف على أنواع الفهارس في SQL Server وكيفية استخدامها عملياً لتحسين سرعة الاستعلامات.

سنتناول في هذا المقرر:

1. كيفية تحسين أداء تعليمات ال SQL عن طريق الفهارس INDEXING بأنواعها المختلفة
2. Transactions
3. Procedures & Functions
4. Triggers

كيفية إنشاء قاعدة المعطيات

1. الطريقة الأولى:



نقوم بفتح البرنامج ومن ثم نختار

Right Click مع Database

ونختار New Database

2. الطريقة الثانية إنشاء القاعدة عبر Commands

يمكن تحديد:

- اسم قاعدة المعطيات
- مسارات الملفات
- حجم الملف
- حجم النمو التلقائي

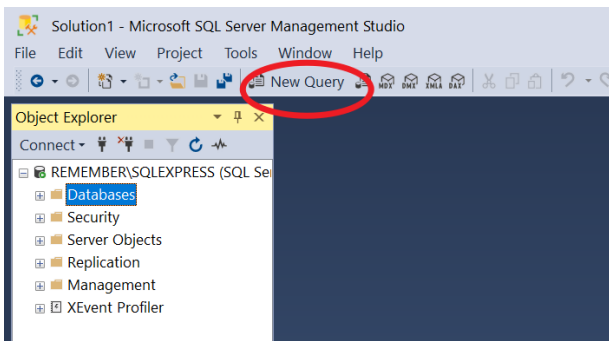
بهدف تنظيم البيانات في مجموعات (File Groups)، الملف ذو اللاحقة MDF هو الأساسي، بينما NDF ملفات إضافية غير محدودة العدد.

أي يمكنك التحكم بدقة بـ:

- حجم النمو التلقائي
- عدد الملفات
- File Groups

- اسم قاعدة البيانات
- أين تخزن ملفاتها على الهارد
- حجم كل ملف

مثال :



لإنشاء تعليمة جديدة:



نكتب الكود هنا وننفذه:

```

CREATE DATABASE MyDB -- الأمر لإنشاء قاعدة بيانات جديدة اسمها MyDB
ON -- يحدد ملفات البيانات
PRIMARY ( -- يحدد الملف الرئيسي (Primary File)
    NAME = MyDB_Main, -- اسم منطقي للملف
    FILENAME = 'C:\SQLData\MyDB.mdf', -- المسار الفعلي للملف البيانات الرئيسي (.mdf)
    SIZE = 50MB, -- الحجم الابتدائي للملف
    MAXSIZE = 500MB, -- أقصى حجم يمكن للملف أن يصل إليه
    FILEGROWTH = 10MB -- مقدار الزيادة في حجم الملف عند امتلائه
),
FILEGROUP DataFiles ( -- يحدد مجموعة ملفات ثانوية (اختياري)
    NAME = MyDB_Extra, -- اسم منطقي للملف الإضافي
    FILENAME = 'D:\SQLData\MyDB_Extra.ndf', -- المسار الفعلي للملف البيانات الثانوي (.ndf)
    SIZE = 100MB,
    FILEGROWTH = 20MB
),
LOG ON ( -- يحدد ملف سجل المعاملات (Transaction Log)
    NAME = MyDB_Log, -- اسم منطقي للملف السجل
    FILENAME = 'E:\SQLLogs\MyDB_Log.ldf', -- المسار الفعلي للملف السجل (.ldf)
    SIZE = 20MB,
    FILEGROWTH = 10MB
)

```

ملاحظة هامة جداً:

قبل تشغيل الكود، تأكد من أن المسارات الموجودة في الكود موجودة على الهارد الخاص بك:

C:\SQLData\
E:\SQLData\
D:\SQLLogs\

ملاحظات:

Database files سواء كانت primary أو secondary هي بالمجمل Database وتوزعها على group files لا يؤثر علينا.

لكن يمكننا تخزين بيانات خارج DB على الهارد كي لا تختلط البيانات، ولكن قد نواجه مشاكل منها:

1. قد ننسى أخذ نسخة احتياطية من البيانات المخزنة على الهارد وعند الاستعادة سيتم فقدها.
2. تصبح عرضة للفايروس بينما لو بقيت ضمن قاعدة المعطيات تكون أقوى حماية.

لذلك يفضل التخزين دوماً على النظام وليس على الهارد.

SQL Server يتعامل مع ثلاثة أنواع من الملفات

1. Primary Data File (MDF)
2. Secondary Data File (NDF)
3. Log File



1. MDF: (الملف الأساسي لقاعدة البيانات) (PRIMARY FILE) ويحتوي:

- الجداول
- البيانات
- schema
- كل بنية القاعدة الأساسية

هو الملف الإجباري الوحيد أي لا يوجد قاعدة بيانات بدون MDF

2. NDF: (الملف الإضافي (الاختياري) (SECONDARY):

يستخدم من أجل:

- توسيع قاعدة البيانات
- توزيع البيانات على أقراص مختلفة
- عند الاحتياج لأداء أعلى
- التعامل مع البيانات الضخمة

يمكن أن يكون لدينا عدة ملفات منه أي يمكن أن يوجد (صفر أو أكثر) والبيانات فعلياً يمكن أن تتوزع بين MDF و NDF حسب ضبطنا للإعدادات.

⚡ ملاحظات

■ هل دائماً البيانات تكون في MDF فقط؟

لا، لأن البيانات ممكن أن تكون إما في MDF أو NDF. أي يمكن أن تتوزع بينهما (بين واحد من ال MDF وواحد أو أكثر من ال NDF)

MDF دائماً يحتوي جزء من البيانات

NDF يحتوي جزء آخر (لو كان موجود)

أي:

- ليس صحيح 100٪ إن "البيانات فقط داخل MDF"
- الصحيح: MDF هو الأساسي، و NDF للتوسعة.



■ هل مسموح أن يكون لدينا فقط ملف MDF واحد فقط وعدة ملفات من ال NDF:

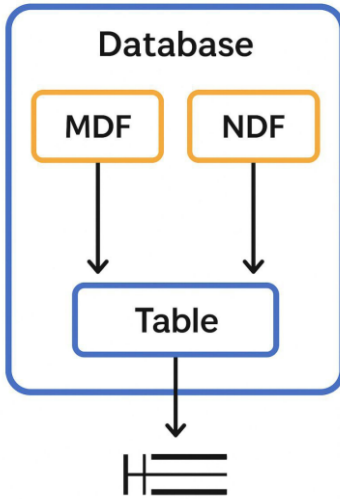
نعم:

MDF واحد فقط دائماً، بينما NDF ممكن يكون: صفر، واحد، أكثر من ملف

لماذا نستخدم NDF Files أساساً؟

قد نستخدمه لتخزين الـ data الموجودة في الأسطر على disk drives مختلفة، كما قد يحسن من الأداء، أو يفيد عند توسعة الـ database (بعد إنشائها) لزيادة حجمها. كما يستخدم عند تخزين ملفات ذات حجوم كبيرة مثل المستندات Documentations.

المخلص:



"الـ MDF هو الملف الأساسي لقاعدة البيانات، ويحتوي على بنية قاعدة البيانات (الجدول، الأعمدة، العلاقات) بالإضافة إلى البيانات نفسها. يمكن إنشاء ملفات NDF إضافية لتخزين البيانات عند الحاجة لتوسيع قاعدة البيانات أو توزيعها على أقراص مختلفة. يجب أن تحتوي كل قاعدة بيانات على ملف MDF واحد فقط، بينما يمكن أن تحتوي على صفر أو أكثر من ملفات NDF."

↪ بالعودة لمثالنا السابق نجد ان :

1. Primary Data File (MDF)

- اسم الملف: MyDB_Main
- مكان التخزين: C:\SQLData\MyDB.mdf
- الحجم الابتدائي: MB50، الحد الأقصى: MB500، نمو تلقائي: MB10
- الدور: الملف الأساسي للقاعدة، يحتوي على الجداول، البيانات، وبنية القاعدة. ضروري دائماً.

2. Secondary Data File (NDF)

- اسم الملف: MyDB_Extra
- مكان التخزين: D:\SQLData\MyDB_Extra.ndf
- الحجم الابتدائي: MB100، نمو تلقائي: MB20
- الدور: ملف إضافي لتوسيع قاعدة البيانات أو تحسين الأداء. اختياري.

3. Log File (LDF)

- اسم الملف: MyDB_Log
- مكان التخزين: E:\SQLLogs\MyDB_Log.ldf
- الحجم الابتدائي: MB20، نمو تلقائي: MB10
- الدور: يسجل كل التعديلات على البيانات قبل تنفيذها، مهم للاستعادة والنسخ الاحتياطي وإدارة المعاملات. ضروري.

ال File Groups هي عبارة عن تقسيم منطقي للملفات الفيزيائية (ndf و mdf) عندها كل مجموعة ملفات يمكنني أن أضعهم ضمن File Group

الخلاصة

- MDF: أساسي، كل قاعدة بيانات تحتاجه.
- NDF: اختياري، للتوسعة أو الأداء.
- LDF: ضروري، لتأمين البيانات وإدارة التعديلات.

بعد تسمية قاعدة المعطيات علينا تحديد أنواع الملفات وهي تقسم لنوعين:

1. Row Data

هو الملف الذي تخزن ضمنه البيانات ويكون نوعين:
Primary أو Secondary

2. Log

حيث يتم تسجيل تعليمات الـ DML أولاً في log،
ثم تنفذ على الـ DB.
DML مثل: insert, delete, update
وهي التعليمات التي تعدل على Database.

■ سؤال: ما الفائدة من وجود Log file؟

- من الممكن أثناء إدخال البيانات أن يحدث انقطاع التيار الكهربائي مما يؤدي إلى انهيار النظام (system) وبذلك نجد مجموعة تعليمات تم تخزينها ضمن ملف ldf من أجل تنفيذ recovery واستعادة ما فقد.
- أي الـ Log File هو دفتر ملاحظات مؤقت مهمته :
تسجيل كل عملية تعديل قبل ما أن تنتفذ فعلياً على قاعدة البيانات.
أي قبل أن نكتب أي شيء داخل الـ Database

■ SQL Server يكتب العملية أولاً داخل ملف الـ LDF ثم يقوم بتنفيذ العمليات على الجدول الحقيقي

فائدته:

1. السبب الأول: حماية البيانات من الضياع :
الـ SQL يسترجع آخر العمليات المكتوبة في الـ Log File يرجع يشغلها تلقائياً وهذا اسمه Recovery.
2. السبب الثاني: تنفيذ عمليات كبيرة بأمان: (Transactions)
3. السبب الثالث: النسخ الاحتياطي (Backup):

■ Log File يستخدم لعمل:

- Point-in-time recovery
- Differential backups
- Transaction log backup

أي يمكنك إرجاع قاعدة البيانات لوقت معيّن بالضبط.

مثال أربيسيزي: تخيل مكتبة يوجد بها موظف يسجل كل الكتب التي يأخذها الناس قبل إعطاء الزبون الكتاب يكتب اسمه واسم الكتاب على دفتر صغير فإذا ضاع الكمبيوتر أو حدث خلل يعود للدفتر ويرى آخر العمليات. هذا هو ملف الـ LDF.

■ مثال :

سنقوم ببناء Table بسيط: (emp) الجدول يحوي:

id, first_name, last_name, gender, salary

سنقوم بعمل Insert

ونلاحظ أن الـ id غير مرتب لذا ستكون القيم المدخلة غير مرتبة، على الرغم من تساوي القيم المدخلة (ID). وعلى الرغم من تحديد العمود كـ identity Column يولد قيماً (1-2-3....) بشكل تسلسلي فقط في حال لم يكن هناك قيم مدخلة.

```
CREATE TABLE EMP1 (
  id INT IDENTITY,
  first_name VARCHAR(255),
  last_name VARCHAR(255),
  gender VARCHAR(10),
  salary INT
);

INSERT INTO EMP1 (first_name, last_name, gender, salary) VALUES ('Ahmed', 'Ali', 'Male', 5000);
INSERT INTO EMP1 (first_name, last_name, gender, salary) VALUES ('Fatima', 'Mohamed', 'Female', 6000);
INSERT INTO EMP1 (first_name, last_name, gender, salary) VALUES ('Omar', 'Hassan', 'Male', 5500);
INSERT INTO EMP1 (first_name, last_name, gender, salary) VALUES ('Khadija', 'Saeed', 'Female', 7000);
INSERT INTO EMP1 (first_name, last_name, gender, salary) VALUES ('Youssef', 'Khalid', 'Male', 4800);

SELECT * FROM EMP1;
```

108 %

Results Messages

	id	first_name	last_name	gender	salary
1	1	Ahmed	Ali	Male	5000
2	2	Fatima	Mohamed	Female	6000
3	3	Omar	Hassan	Male	5500
4	4	Khadija	Saeed	Female	7000
5	5	Youssef	Khalid	Male	4800

أيضا من العوامل المهمة في السرعة هي الـ (MEMORY DISK) حيث انه يجب لاستخدام الداتا ان ترفع الـ DATA BLOCK على الـ RAM

والDATA BLOCK :

تحتوي مجموعة ال RECORDS فعندما نقوم مثلاً بعملية ال SELECT سيقوم بالبحث عن الداتا وسيقرأها من ال DISK ومن ثم سيقوم بتحميلها على ال MEMORY

ملاحظة ل التذكير :

يوجد لدينا نوعين من التحميل وهما :

1. Physical read: تحميل صفحة من القرص (DISK) إلى ذاكرة التخزين المؤقت وهذه العملية بطيئة لأن الصفحة غير موجودة في الذاكرة.

2. Logical read: (سرع بكثير لأنه يقوم بقراءة صفحة موجودة في ال MEMORY

- ولذلك نسعى أن تكون جميع الداتا لدينا موجودة على ال RAM حتى يكون التعامل معها سريع
- أول استعمال سنقوم به سيكون بطيء لأنه سيتم القراءة فيزيائية

Execution plan

Optimizer: هو الذي يقوم بالبحث عن أفضل طريقة لتنفيذ الاستعلام وهو مصمم بتقنيات الذكاء الصناعي للبحث عن أقصر طريق يبدأ بالبحث في الجدول في حال كان يملك indexes وفي حال لم يجد أي وسيلة مساعدة للتسريع يقوم بعملية Table Scan وهي أسوأ أنواع جلب الداتا لأنه يتطلب مسح الجدول كامل تظهر بال ctrl+l

■ سؤال : ماهو الفرق بين ال Execution Plan وال Query Optimizer :

- Query Optimizer: يختار أفضل خطة ممكنة لتنفيذ الاستعلام
- Execution Plan: هو الخطة التي اختارها ال Optimizer ونفذها SQL Server

مثال للتوضيح:

أنت كتبت استعلام: 'SELECT * FROM Orders WHERE OrderDate = '2023-01-01'

ال Optimizer فكر: هل أستخدم فهرس على OrderDate؟ أم أعمل Table Scan؟

قرر أن الفهرس مناسب فاختار Index Seek

ال Execution Plan سيظهر أن SQL Server استخدم Index Seek على OrderDate

Indexing

لتبسيط فكرة الـ Index لنفترض أنك اشتريت كتاباً فيه 3500 صفحة عن تعليمات SQL وأنت تريد تعليمة محددة.

- بدون فهرس <= ستضطر لتصفح كل الكتاب.
- مع فهرس <= تذهب مباشرة للصفحة المطلوبة.

وهذا هو مبدأ الفهارس indexing

مع العودة لمثالنا السابق نقوم باستخراج الـ EXECUTION PLAN

يظهر لدينا الـ TABLE SCAN

لوقمنا باختيار جلب لجميع البيانات التي لدينا أي نقوم بعمل

- Select * from emp 1

SELECT * FROM EMP1;

id	first_name	last_name	gender	salary
1	Ahmed	Ali	Male	5000
2	Fatima	Mohamed	Female	6000
3	Omar	Hassan	Male	5500
4	Khadija	Saeed	Female	7000
5	Youssef	Khalid	Male	4800

SELECT * FROM EMP1;

Query 1: Query cost (relative to the SELECT * FROM EMP1

Operation	Cost	Rows
Table Scan [EMP1]	Cost: 100 %	0.000s
SELECT	Cost: 0 %	5 of 5 (100%)

يظهر لدينا الـ TABLE SCAN أي انه قام بقراءة جميع البيانات التي لدينا حتى يصل الى الاستعلام المطلوب

لتشغيل الـ Execution Plan نكتب الاختصار Ctrl + M



- لكن هل سيختلف اذا قمنا باستخدام الشرط WHERE أي يصبح شكل التعليمة كما يلي :

SELECT * FROM EMP1
where salary = 4800;

Query 1: Query cost (relative to the batch): 10
SELECT * FROM [EMP1] WHERE [salary]=@1

Operation	Cost	Rows
Table Scan [EMP1]	Cost: 100 %	0.000s
SELECT	Cost: 0 %	1 of 1 (100%)

نلاحظ انه في كلا الحالتين تم مسح كل الـ

Rows والاختلاف فقط في Result set.

وذلك لانه بكلا الحالتين قام بقراءة جميع الداتا التي

يحويها الجدول أي قام بعمل TABLE SCAN

■ لهذا سنقوم باستخدام ال فهرس التي تدعمها ال SQL وهي نوعين :

1. CLUSTERED INDEX

2. NON-CLUSTERED INDEX

وسنقوم بدراستها في المحاضرة القادمة

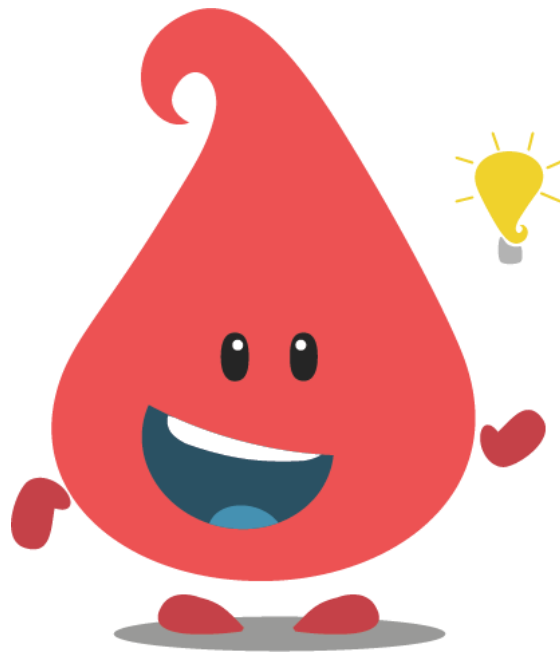
الفرق بين ال SEEK وال SCAN :

SCAN TABLE: قراءة كل الصفوف او صفحات الفهرس

SEEK: الوصول المباشر الى الصفوف المطلوبة عبر الفهرس

" أن تضيء شمعة صغيرة خير لك من أن

تنفق عمرك تلعن الظلام "



-- النهاية --